Toronto Metropolitan University

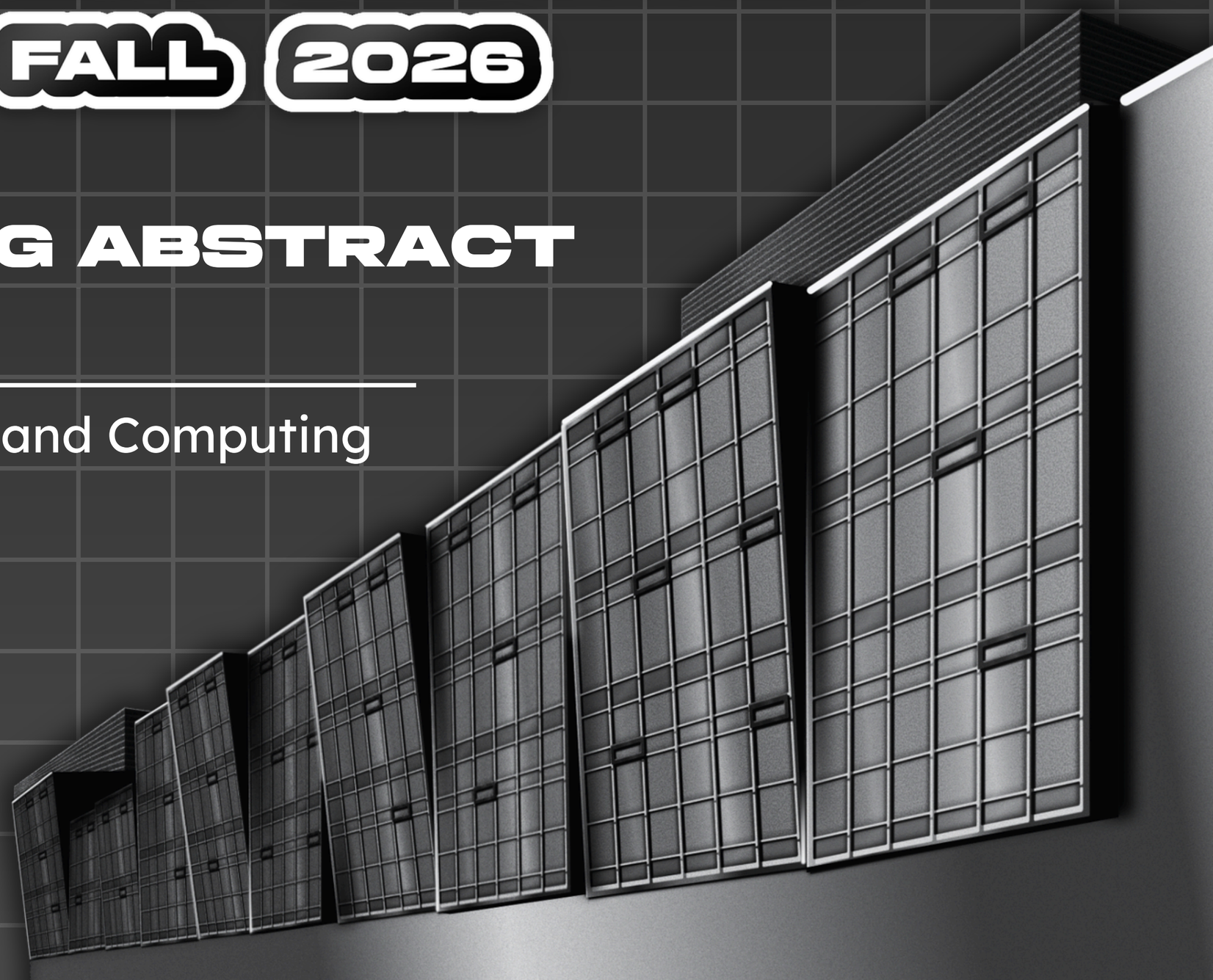
Metropolitan Undergraduate Engineering Society

TORONTO METROPOLITAN UNIVERSITY ENGINEERING COMPETITION FALL 2026

PROGRAMMING ABSTRACT

NOVEMBER 7 & 8, 2026

George Vari Engineering and Computing Centre



Metropolitan Engineering Competition 2026
Programming
Competition Package

CONTENTS

1. Introduction 01
2. Guidelines 02
    2.1 - Allowed Resources 02
    2.2 - Question Periods 02
3. Competition Description 02
    3.1 - The Challenge 03
    3.2 - Assessment 05
4. Competition Deliverables 06
    4.1 - Code and README File 06
    4.2 - Presentation 07
    4.3 - Plagiarism 07

1. INTRODUCTION

Welcome to the 21st Metropolitan Engineering Competition (MEC 2026)!

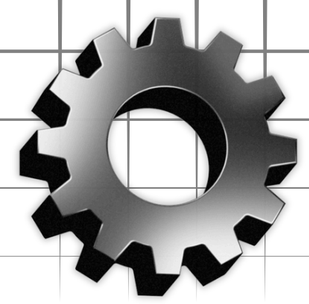
This year's competition theme is Feeding the Future: Designing Smart Infrastructure for Urban Food Security.

For the Programming Competition, your team will be developing an Urban Food Crisis Response and Routing System. The goal is to create a program that helps communities respond to food shortages by deciding which emergencies should be addressed first, where food supplies can be obtained, and how those supplies can be delivered as efficiently as possible.

Food emergencies can happen for many reasons, and responding to them is not always straightforward. Some locations may have more urgent needs than others, food supplies may be limited, delivery vehicles may not be available, and unexpected problems such as road closures can make deliveries more difficult.

Your program will need to consider these challenges and make decisions based on the information provided.

To make the competition more realistic, teams will work with two different datasets. The first represents a small section of downtown Toronto, while the second represents a more open town or city with fewer food supply locations and resources. Both datasets will include unexpected situations that may affect how emergencies are handled.



The challenge is to build a universal system that can work in both environments, adapt to changing conditions, and make fair and efficient decisions.

This competition is an opportunity to apply your programming knowledge, problem-solving skills, and creativity to a real-world engineering challenge.

2. GUIDELINES

2.1 - Allowed Resources

The competitors can bring the following equipment:

- Any electronic material stored on USBs, external hard drives, and any other sort of storage devices
- Any textbooks, course notes, or other reference material
- Each team member is allowed one computer and access to the internet

Use of Generative AI:

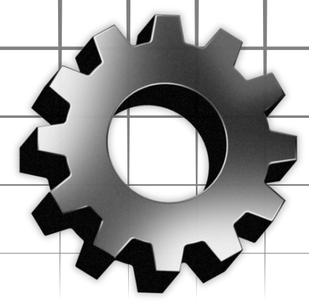
- Teams may use generative AI tools in this competition as long as the use is properly referenced and cited following the posted citation guidelines. Any AI-generated content that is used must be reviewed and meaningfully modified by the competitors.
- Examples of acceptable use include brainstorming ideas, researching background information, debugging code, or assisting with documentation and report formatting. Examples of unacceptable use include copying full solutions, submitting unedited AI-generated code, or using AI to write presentation scripts.
- Competitors are responsible for ensuring that their AI use aligns with competition expectations and reflects their own technical understanding of the solution.

2.2 - Question Periods

- There will be a period for questions after the challenge is presented by the Competition Coordinators. During this time, competitors can ask for clarification on the problem statement, rules, scoring, etc.
- If you need help during the build phase, you can:
  - Ask the MEC team member in your room to assist you
  - Press the “Help” button on the MEC shop website
  - Email mec.vpcompetitions@mues.ca for any competition-related questions.

3. COMPETITION DESCRIPTION

Imagine a hospital, shelter, or community centre running low on food. Supplies may be available elsewhere, but getting them to the people who need them is not always simple.



Some emergencies are more urgent than others. Food supplies and delivery vehicles are limited, and roads may suddenly become unavailable. In these situations, making the right decisions quickly can make a significant difference.

For this competition, your team will develop an Urban Food Crisis Response and Routing System that helps manage these situations.

Your system should be able to receive emergency food requests, determine their priority, identify suitable food supplies and delivery vehicles, calculate efficient delivery routes, and track the progress of each emergency.

It should also respond to changing conditions, including road closures, vehicle breakdowns, limited food supplies, and multiple emergencies happening at the same time.

To test how adaptable your solution is, you will work with two different environments.

The first is a small section of downtown Toronto, where there may be several possible routes and food supply locations. The second is a more open, less-serviced town or city, where food sources are more limited and reaching certain locations may be more difficult.

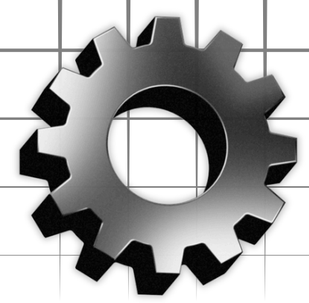
Your goal is to build a system that can handle both situations without needing to be redesigned for each one.

A live website dashboard is optional. Teams may implement their system using a command-line interface, desktop application, or another suitable interface, provided the required functionality can be demonstrated and evaluated. A functional live website dashboard is worth up to five (5) points within the official 100-point judging rubric.

The focus of this competition is not just finding the shortest route. It is about making practical decisions when time, resources, and available options are limited.

3.1 - The Challenge

Your task is to build a universal working Urban Food Crisis Response and Routing System that can manage emergency food deliveries in different city environments. The system must take emergency requests, determine which requests should be handled first, assign suitable food supplies and delivery vehicles, and calculate efficient routes. The city road network should be treated as a weighted graph, where locations are represented as nodes and roads are represented as edges. Each road will have a cost, such as distance or estimated travel time. Your program must also account for changing conditions and unexpected events.



Your solution should address the following requirements:

- **Crisis submission:** Allow users to enter emergency requests, including the location, crisis type, location category, number of people affected, food quantity needed, refrigeration requirements, severity level, delivery deadline, and submission time. The system should check for missing or invalid information before processing a request.
- **Priority scoring:** Develop a scoring system that determines how urgent each emergency is. Your scoring system should consider factors such as severity, number of people affected, food shortage, delivery deadlines, location type, and relevant travel or response-time considerations. Teams should be able to explain how their scoring system works and why certain emergencies are prioritized over others.
- **Dynamic priority queue:** Use a heap-based priority queue to organize emergency requests based on urgency. The system should update priorities when conditions change and include a waiting-time adjustment so lower-priority requests are not ignored indefinitely. Teams should also determine how requests with equal priority are handled.
- **Route planning:** Use Dijkstra's or A* shortest-path algorithm to calculate efficient delivery routes using the provided road network. The system must account for road closures and other disruptions, find alternative routes when possible, and clearly indicate when a delivery route is unavailable.
- **Resource matching:** Use a greedy approach to assign suitable food supplies and delivery vehicles to emergency requests. The system should consider available food stock, vehicle capacity, refrigeration requirements, delivery urgency, and route availability. It must also prevent the same resources from being assigned to multiple emergencies at the same time.
- **Crisis state machine:** Track each emergency through appropriate states, including Open, Prioritized, Resource Assigned, In Progress, Re-Routing, Delivered, Resolved, Delayed, and Failed. Teams must implement valid state transitions based on the situation. For example, Re-Routing should occur when a route becomes unavailable rather than being required for every delivery. Each status change must be recorded with a timestamp and an explanation of what happened.
- **System events:** Use a publish/subscribe event bus for crisis-created, priority-changed, route-updated, resource-depleted, road-closed, vehicle-breakdown, and crisis-resolved events. A WebSocket-powered website can show these changes live.

Competitors will receive the first dataset on Day 1 to develop and test their system. A second dataset will be provided on Day 2 to evaluate how well the same system adapts to a different environment. Both datasets will follow the same file structure and formatting, including file names, headers, and data organization. Teams should design their programs to process either dataset without requiring changes to the core program logic.

Teams must develop their core system using Dataset 1. Dataset 2 will be used to evaluate the adaptability of the submitted solution. Teams may select or load the second dataset during evaluation but may not modify their submitted source code after the development deadline.

Map of Downtown Toronto from Dataset One:



The Competition Organizers will provide the data necessary to evaluate the required system functionality.

The competition materials will identify the road-network data, available food supplies, delivery vehicles, emergency requests, and disruption events. These may be supplied in separate files or within a common dataset structure.

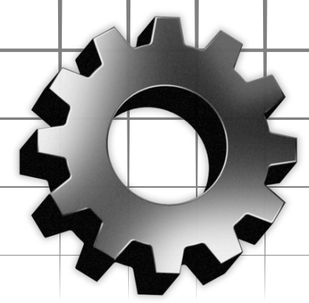
Each dataset will use a documented, consistent format. Teams must be able to load and process the provided information without modifying their submitted core algorithms.

The Competition Coordinators will explain the dataset structure and identify any required assumptions during the official competition briefing.

3.2 - Assessment

Judges will also consider how well the system performs across both datasets. Teams should demonstrate that their solution can adapt to different environments, handle limited resources, and respond to unexpected events such as road closures and vehicle breakdowns.

A strong solution should not only work when conditions are ideal, but should also be able to explain what happens when a delivery cannot be completed or when an alternative route or resource is needed.



Judges will assess projects using the following criteria:

- **Algorithmic Effectiveness:** How well the system prioritizes emergencies, finds efficient delivery routes, and assigns available food and vehicles to those in need.
- **System Design and Reliability:** How effectively the system tracks emergencies, responds to changing situations, and handles challenges such as road closures, food shortages, and vehicle breakdowns.
- **User Experience:** How easy the system is to understand and use, including how clearly it displays emergency priorities, delivery routes, available resources, and live updates.
- **Innovation and Scalability:** How creatively the team approaches the challenge and how well their system could adapt to larger cities, more emergencies, and changing resource availability.

The goal is to encourage competitors to use their programming skills and creativity to build a practical system that helps cities respond to food emergencies more quickly, efficiently, and reliably.

4. COMPETITION DELIVERABLES

Teams in the Programming Competition are required to develop their project within the allotted competition time. The first phase consists of program development, and the second phase consists of a formal presentation and demonstration to the judging panel.

Teams will have eight (8) hours to complete the development of their solution, including coding, testing, and documentation, before submitting their final code package at the end of the project period.

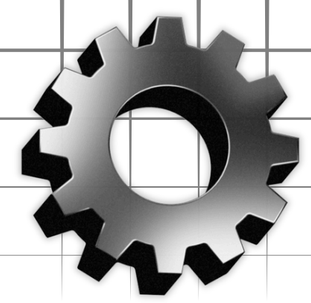
All deliverables must be submitted to the Programming Competition Coordinators by the specified deadlines. Late submissions for any component will result in the deductions outlined in the rulebook.

Note: You can use any programming language you are comfortable with as long as you have the correct environment set up.

4.1 - Code and README File

All project code must be compressed into a .zip file and submitted **by the end of the project period**.
The folder must include a README.md file containing the following:

- Administrative information: team name, competitor names, and project title
- A brief overview of the program’s purpose and functionality
- Clear instructions to compile and run the program for evaluation
- A list of dependencies or libraries used (for example: Python, TensorFlow, Pandas)
- Proper citations for any datasets, frameworks, or generative AI tools used
- Short comments in key sections of code explaining logic and structure



The naming convention should follow:

TeamName_Programming_Code.zip

4.2 - Presentation

As a key component of the submission, the presentation allows teams to demonstrate their solution and explain its technical design. The presentation file must be submitted along with the code at the end of the design phase. Teams should include screenshots, results, and visualizations that demonstrate how their system performs using the provided competition data. Where applicable, teams should explain how their system handles the different environments represented in the dataset, including changes in road availability, resource limitations, and emergency priorities. Teams should also be prepared to demonstrate how their program responds to unexpected situations and explain the decisions made by their algorithms.

All team members must be present. Absence of a member will result in a 25-point deduction, unless previously approved by the VPs and Chairs.

Guidelines:

- Presentations must be no longer than 10 minutes, followed by a 5-minute question period.
- Walk the judges through the implementation, including technologies used and any technical challenges faced.
- Demonstrate the submitted solution using the competition dataset or datasets specified by the Competition Coordinators. Teams must be prepared to demonstrate how the same program processes Dataset 2 without modifying its submitted source code.
- Highlight the program's engineering approach, algorithmic decisions, practical applications, and contributions to efficient and sustainable food distribution.
- Keep visuals clear and concise; charts, flow diagrams, and screenshots are encouraged.

Presentations must be submitted as a PDF with the naming convention:

TeamName_Programming_Presentation.pdf

4.3 - Plagiarism

Use of external resources, libraries, or generative AI tools is permitted provided that all sources are properly cited in the code comments, README, and presentation. You may NOT use a library or open-source code to solve everything for you.

Reusing large portions of unedited external code or datasets without acknowledgment, or presenting AI-generated material as original work, will be treated as plagiarism and may result in

disqualification. Judges may ask competitors to explain specific code segments; failure to do so may lead to point deductions.

Teams are expected to maintain academic integrity and uphold the spirit of fairness and innovation central to the Metropolitan Engineering Competition.